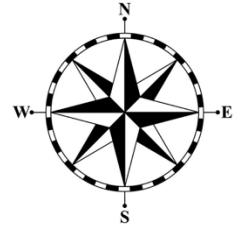


Testing in Python

Kevin Cunningham





Plan for Session

- ❑ What is pytest?
- ❑ Writing tests
- ❑ Test Fixtures
- ❑ Parameterized Tests
- ❑ Ignoring Tests
- ❑ Testing for exceptions

What is pytest?

❑ Testing library for Python

- one of the most popular

- ❑ see <https://docs.pytest.org/en/latest/>



- builds on top of / extends unittest

- ❑ provides an extended set of facilities

❑ Can be used for different levels of test

- unit, integration, system/e2e

- most typically used for unit

- used by Dropbox, Mozilla etc. as Python testing framework

❑ Has support in IDEs such as PyCharm

- allowing for red bar, green bar development

Simple pytest Example



❑ Need something to test

■ May be a function

```
def add(x, y):  
    return x + y
```

■ Can be a class

- ❑ Calculator class in `calculator.py` file
- ❑ has the concept of the current total
- ❑ can have a current value entered
- ❑ the action to apply to the current value

```
class Calculator:  
    def __init__(self):  
        self.current = 0  
        self.total = 0  
  
    def set(self, value):  
        self.current = value  
  
    def add(self):  
        self.total += self.current  
  
    def sub(self):  
        self.total -= self.current  
  
    def total(self):  
        return self.total
```

Writing a Test



- ❑ Place tests in separate file / module
- ❑ Test defined in a file `test_calculator.py`
 - therefore, a separate *module*
 - so will need to import from `calculator` module
- ❑ pytest uses conventions to find tests:
 - search for `test_*.py` or `*_test.py` files.
 - From those files, collect test items:
 - ❑ `test_` prefixed test functions
- ❑ IDE typically provides support for creating suitable test files



Writing a Test



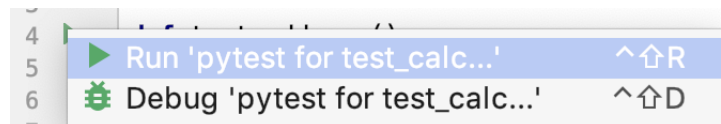
- A simple test
 - note use of descriptive test function name
 - makes it easier to understand test reports
- Run using PyTest from  command line or via IDE

```
from calculator import add
```

```
def test_add_zeros():  
    result = add(0, 0)  
    assert result == 0
```

```
def test_add_one_and_zero():  
    result = add(1, 0)  
    assert result == 1
```

```
def test_add_zero_and_one():  
    result = add(0, 1)  
    assert result == 1, "should be one"
```



Writing a Test

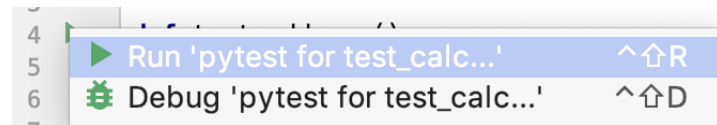


- Can also test classes in exactly the same way

```
from calculator import Calculator

def test_add_one():
    calc = Calculator()
    calc.set(1)
    calc.add()
    assert calc.total == 1
```

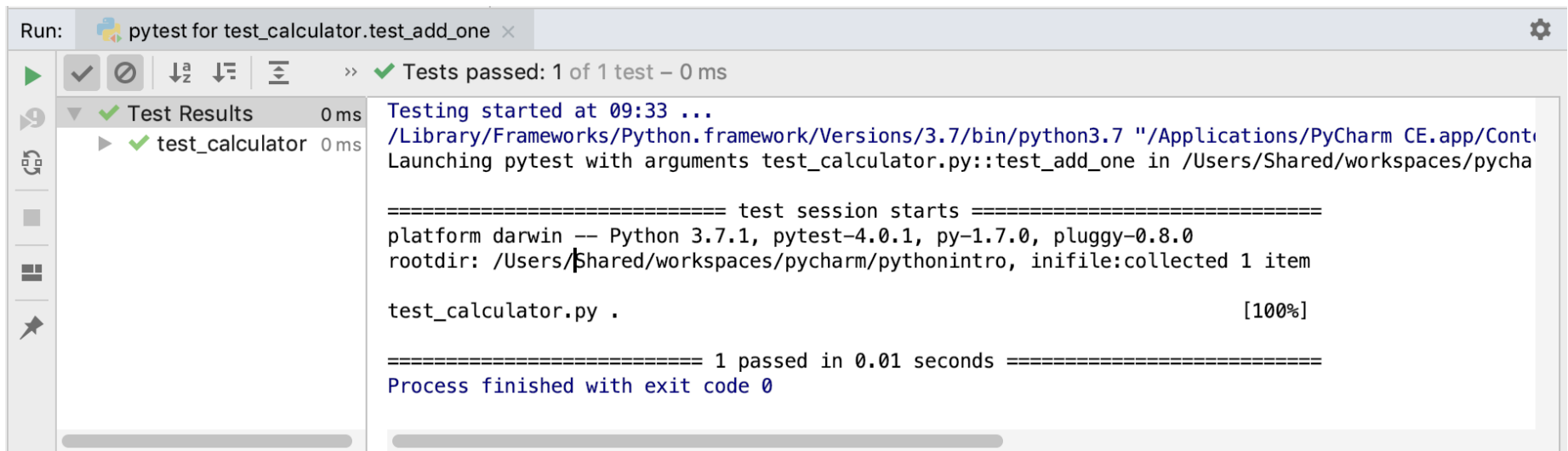
- Again, can run from command line or via IDE
 - Using PyTest



Running a Test



- ❑ In IDE can see result of tests
 - and explore test reports

A screenshot of the PyCharm IDE's 'Run' window. The window title is 'Run: pytest for test_calculator.test_add_one'. It shows a successful test run with a green checkmark and the text 'Tests passed: 1 of 1 test - 0 ms'. Below this, a tree view shows 'Test Results' with a green checkmark and 'test_calculator' with a green checkmark. The main text area displays the test output, including the start time, the command used to launch pytest, the test session details, and the final result: '1 passed in 0.01 seconds'.

```
Run: pytest for test_calculator.test_add_one x
>> ✓ Tests passed: 1 of 1 test - 0 ms

Test Results 0 ms
  ✓ test_calculator 0 ms

Testing started at 09:33 ...
/Library/Frameworks/Python.framework/Versions/3.7/bin/python3.7 "/Applications/PyCharm CE.app/Contents/Python/bin/python3.7" -m pytest test_calculator.py::test_add_one
Launching pytest with arguments test_calculator.py::test_add_one in /Users/Shared/workspaces/pycharm/pythonintro

===== test session starts =====
platform darwin -- Python 3.7.1, pytest-4.0.1, py-1.7.0, pluggy-0.8.0
rootdir: /Users/Shared/workspaces/pycharm/pythonintro, inifile:collected 1 item

test_calculator.py . [100%]

===== 1 passed in 0.01 seconds =====
Process finished with exit code 0
```




Test Fixtures

- ❑ May need to run some behaviour
 - before or after each test or
 - before or after a group of tests in a module
 - to provide resource for a test
- ❑ Such code is known as a fixture
 - may set up scenario for test
 - supply an object to be test
 - perform post test housekeeping etc.
- ❑ Two styles
 - naming conventions
 - PyTest Python decorators



Test Fixtures as Functions

▣ Based on naming conventions

```
from calculator import add

def setup_module():
    print('module set up')

def teardown_module():
    print('module teardown')

def setup_function():
    print('function set up')

def teardown_function():
    print('function teardown')

def test_add_one_and_one():
    result = add(1, 1)
    assert result == 2
```

```
module set up
function set up
PASSED [100%]
function teardown
module teardown
```

Test Fixtures Using Decorators



□ Can use pytest fixture decorator

- have format @<name>
- to indicate fixtures for tests

```
import pytest
from calculator import Calculator
```

```
@pytest.fixture
def calculator():
    """Returns a Calculator instance"""
    print('calculator fixture')
    return Calculator()
```

```
calculator fixture
calculator fixture
calculator fixture
calculator fixture
```

```
def test_initial_value(calculator):
    assert calculator.total == 0
```

```
def test_add_one(calculator):
    calculator.set(1)
    calculator.add()
    assert calculator.total == 1
```

```
def test_subtract_one(calculator):
    calculator.set(1)
    calculator.sub()
    assert calculator.total == -1
```

```
def test_add_one_and_one(calculator):
    calculator.set(1)
    calculator.add()
    calculator.set(1)
    calculator.add()
    assert calculator.total == 2
```

Ignoring Tests



- May want to ignore tests
 - particularly during development
- To ignore a test
 - decorate with `@pytest.mark.skip`

```
@pytest.mark.skip(reason='not implemented yet')  
def test_calculator_multiply(calculator):  
    calculator.multiply(2, 3)  
    assert calculator.total == 6
```

- Reported on in IDE / pytest report

Test Results	0 ms
test_calculator	0 ms
test_initial_value	0 ms
test_add_one	0 ms
test_subtract_one	0 ms
test_add_one_and_one	0 ms
test_calculator_add_operation	0 ms
test_calculator_multiply	0 ms

Parameterized Tests



- Common to run the same tests multiple times with several different input values
 - can greatly reduce the number of tests needed
- Such tests are referred to as parametrised tests
 - specified via `@pytest.mark.parametrize` decorator
 - with values to apply provided to decorator

```
@pytest.mark.parametrize('input1,input2,expected', [
    (3, 1, 4),
    (3, 2, 5),
])
def test_calculator_add_operation(calculator, input1, input2, expected):
    calculator.set(input1)
    calculator.add()
    calculator.set(input2)
    calculator.add()
    assert calculator.total == expected
```

Testing for Exceptions



- ❑ Can write tests that verify that an exception was raised
- ❑ In PyTest use the with statement and `pytest.raises`

```
import pytest

def test_current_account_overdraft():
    current_account = CurrentAccount(0.0)
    with pytest.raises(accounts.BalanceError):
        current_account.withdraw(200.0)
```

Questions?

